Day 6 — Finance Analytics Service PoC

Problem statement

The PoC helps FP&A analysts and finance controllers compare budget against actual expenditure, identify high-priority expense exceptions, analyse trends, and drill down to individual transactions. It combines the Week 1 backend, API, SQL, caching, testing, observability, and statistical-reasoning topics into one executable project.

Completed deliverable

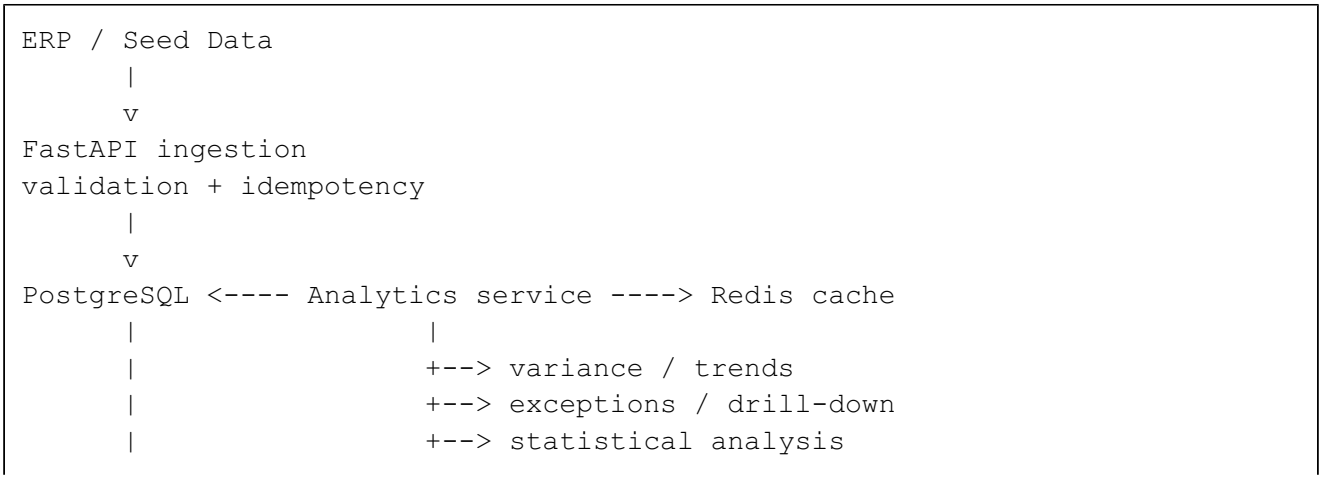
Download the complete Day 6 repository

The repository includes:

- FastAPI endpoints for variance, trends, exceptions, drill-down, ingestion, and statistical analysis.
- PostgreSQL schema, constraints, indexes, CTEs, analytical SQL, and window functions.
- Redis cache implementation with TTL and version-based invalidation.
- SQLite and in-memory-cache profile for execution without Docker.
- Deterministic synthetic data containing 8 departments, 24 cost centres, 30 vendors, 288 budgets, and 2,330 expenses.
- Transactional, idempotent ingestion using an idempotency key and canonical payload hash.
- Stable cursor pagination using the complete sort tuple.
- Structured JSON logs, correlation IDs, timing headers, health checks, operational metrics, and consistent error envelopes.
- Welch's t-test, 95% confidence interval, Cohen's d, interpretation, and limitations.
- Docker Compose, seed scripts, API examples, tests, benchmarks, demo scripts, and interview questions.

The implementation uses FastAPI dependency injection and response models for resource and contract management, with SQLAlchemy sessions defining database transaction boundaries. (FastAPI)

Architecture



```
      |
Structured logs + correlation IDs + health + metrics
```

Executed results

These are measured local results, not invented production metrics:

Metric	Result
Tests	13 passed
Seeded expenses	2,330
Budget reconciliation error	0.00
Actual reconciliation error	0.00
Direct analytical SQL p50	1.5992 ms
Warm-cache p50	0.0778 ms
Measured p50 speed-up	20.55×
Warm-cache hit ratio	99.67%
Exception candidates	834
Exception spend share	56.52%
Welch 95% CI for mean difference	4,836.35–8,851.39

The latency measurements used SQLite and the real in-process TTL-cache implementation. Docker was unavailable in the execution environment, so PostgreSQL and Redis latency is deliberately left as a full-stack benchmark step rather than being estimated.

Main documents

- Day 6 mentor guide
- Repository README and setup
- Measured metrics
- Validation report
- Three-minute and extended demo scripts
- API examples
- Likely interviewer questions
- Actual generated API outputs

Fastest local execution

```
unzip day6_finance_analytics.zip
cd day6_finance_analytics

python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

```
export DATABASE_URL=sqlite:///./finance.db
export REDIS_URL=memory://
export AUTO_CREATE_SCHEMA=true

python scripts/seed_db.py
pytest
uvicorn app.main:app --reload
```

Then open `http://localhost:8000/docs` or call:

```
curl -i http://localhost:8000/v1/analytics/variance
```

Day 6 DSA Track — Stack

1. Stack fundamentals

A **stack** follows **LIFO**:

> Last In, First Out

The most recently added item is removed first.

```
stack = []

stack.append("A")
stack.append("B")
stack.append("C")

print(stack.pop())  # C
print(stack[-1])    # B: inspect top without removing
```

Typical operations:

Operation	Python	Complexity
Push	`stack.append(x)`	O(1) amortized
Pop	`stack.pop()`	O(1)
Peek	`stack[-1]`	O(1)
Check empty	`if not stack:`	O(1)

Avoid removing from the beginning of a Python list:

```
stack.pop(0)  # O(n), not a stack operation
```

---

2. Recognition signals

Consider a stack when the problem contains one or more of these signals:

Nested or paired structures

Examples:

- Parentheses
- XML/HTML tags
- Nested expressions
- Function calls
- Directory paths

Common problems:

- Valid parentheses
- Decode string
- Simplify path
- Evaluate expressions

Reverse-order processing

You encounter something now, but must process it only after later information becomes available.

Examples:

- Undo the most recent action
- Backtracking through browser history
- Reversing operations
- Removing the latest unmatched character

Nearest previous or next element

The problem asks for:

- Next greater element
- Previous smaller element
- Nearest warmer day
- First larger value on the right
- Largest rectangle in a histogram

These often indicate a **monotonic stack**.

State restoration or undo

Each operation changes the current state, but you may need to restore the previous state.

Examples:

- Text editor undo
- Browser back
- Transaction rollback
- Game move history

- Nested configuration scopes

Parsing

A stack is useful when parsing expressions such as:

```
3 + 2 * (4 - 1)
```

Stacks can hold:

- Operators
- Operands
- Parentheses
- Partial results
- Parsing states

---

3. Non-monotonic stack

A regular or non-monotonic stack does not require its elements to remain sorted.

Example: validating parentheses.

```
def is_valid_parentheses(text: str) -> bool:
    pairs = {
        ")": "(",
        "]": "[",
        "}": "{",
    }

    stack: list[str] = []

    for char in text:
        if char in "([{":
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False

    return not stack
```

The stack may contain:

```
[, (, {
```

There is no increasing or decreasing requirement. It simply represents unresolved opening brackets.

---

4. Monotonic stack

A monotonic stack maintains elements in sorted order.

There are two common forms:

Monotonically increasing stack

Values increase from bottom to top.

```
[2, 5, 8, 10]
```

When a smaller value arrives, larger values may be removed.

Useful for:

- Previous smaller element
- Next smaller element
- Histogram boundaries

Monotonically decreasing stack

Values decrease from bottom to top.

```
[10, 8, 5, 2]
```

When a larger value arrives, smaller values may be removed.

Useful for:

- Next greater element
- Stock span
- Daily temperatures

The important insight is that popped elements have just found the first element that resolves their question.

Medium problem: Daily Temperatures

Problem statement

Given a list of daily temperatures, return a list where each position contains the number of days until a warmer temperature.

If no warmer day exists, return `0` for that position.

Example

```
Input:
[73, 74, 75, 71, 69, 72, 76, 73]
```

```
Output:
[1, 1, 4, 2, 1, 1, 0, 0]
```

Explanation:

- Day 0: `73 → 74`, wait 1 day
- Day 1: `74 → 75`, wait 1 day
- Day 2: `75 → 76`, wait 4 days
- Day 3: `71 → 72`, wait 2 days
- Day 4: `69 → 72`, wait 1 day
- Day 5: `72 → 76`, wait 1 day
- Day 6: no warmer day
- Day 7: no warmer day

---

5. Recognition signals

The wording contains several monotonic-stack signals:

- For every element
- Find a future element
- It must be strictly greater
- Find the first qualifying element
- Return the distance between indices
- A brute-force solution repeatedly scans the same suffix

This is effectively:

> For every temperature, find the next greater temperature to its right.

That strongly suggests a monotonic decreasing stack.

---

6. Brute-force reasoning

For every day:

1. Start scanning from the following day.
2. Stop when a warmer temperature is found.
3. Store the difference between the two indices.
4. If no warmer day exists, leave the answer as zero.

Pseudocode

```
create result array filled with zero

for each day i:
    for each later day j:
        if temperature[j] > temperature[i]:
```

```
            result[i] = j - i
            break

return result
```

Python brute-force solution

```
def daily_temperatures_brute_force(
    temperatures: list[int],
) -> list[int]:
    result = [0] * len(temperatures)

    for current_day in range(len(temperatures)):
        for future_day in range(current_day + 1, len(temperatures)):
            if temperatures[future_day] > temperatures[current_day]:
                result[current_day] = future_day - current_day
                break

    return result
```

Why it is inefficient

For a decreasing sequence:

```
[90, 80, 70, 60, 50]
```

Every element scans most of the remaining array and never finds an answer.

The number of comparisons approaches:

```
(n - 1) + (n - 2) + ... + 1
```

Therefore:

- Time: **O(n²)**
- Space: **O(1)** excluding the output

---

7. Optimized reasoning

Instead of solving each day independently, maintain a stack of days that have not yet found a warmer future temperature.

The stack stores **indices**, not only temperatures.

We need indices because the result is:

```
current_index - previous_index
```

Stack invariant

Temperatures corresponding to stack indices remain monotonically decreasing:

```
temperatures[stack[0]] >=
temperatures[stack[1]] >=
temperatures[stack[2]]
```

Suppose the stack contains temperatures:

```
[75, 71, 69]
```

When `72` arrives:

- `72 > 69`: resolve the day containing `69`
- `72 > 71`: resolve the day containing `71`
- `72 < 75`: stop

Then push `72`.

Key insight

When the current temperature is greater than the temperature at the stack top:

```
temperatures[current_day] > temperatures[stack[-1]]
```

the current day is the first warmer day for the stack-top day.

Why the first?

Because every day between them was processed earlier and failed to pop that index.

---

8. Optimized pseudocode

```
create result array filled with zero
create an empty stack of indices

for each current index:
    while stack is not empty
          and current temperature is greater than
              temperature at stack top:

        previous index = pop stack
        result[previous index] = current index - previous index

    push current index onto stack

return result
```

---

9. Python solution

```
from collections.abc import Sequence


def daily_temperatures(
    temperatures: Sequence[int],
) -> list[int]:
    """
    Return the number of days until a warmer temperature.

    Args:
        temperatures:
            Sequence of daily temperatures.

    Returns:
        A list where result[i] is the number of days after day i
        when a warmer temperature occurs. If none exists, result[i]
        remains zero.
    """
    result = [0] * len(temperatures)

    # Stores indices whose next warmer day has not yet been found.
    unresolved_days: list[int] = []

    for current_day, current_temperature in enumerate(temperatures):
        while (
            unresolved_days
            and current_temperature
            > temperatures[unresolved_days[-1]]
        ):
            previous_day = unresolved_days.pop()
            result[previous_day] = current_day - previous_day

        unresolved_days.append(current_day)

    return result
```

Example execution

```
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]

print(daily_temperatures(temperatures))
```

Output:

```
[1, 1, 4, 2, 1, 1, 0, 0]
```

---

10. Dry run

Input:

```
[73, 74, 75, 71, 69, 72]
```

Day	Temperature	Stack before	Action	Result updates
0	73	`[]`	Push 0	None
1	74	`[0]`	Pop 0, push 1	`result[0] = 1`
2	75	`[1]`	Pop 1, push 2	`result[1] = 1`
3	71	`[2]`	Push 3	None
4	69	`[2,3]`	Push 4	None
5	72	`[2,3,4]`	Pop 4, pop 3, push 5	`result[4]=1`, `result[3]=2`

Final unresolved stack:

```
[2, 5]
```

Those positions have no known warmer day yet, so their answers remain zero.

Intermediate result:

```
[1, 1, 0, 2, 1, 0]
```

---

11. Correctness reasoning

The algorithm is correct because of three conditions.

Every unresolved day is stored

When a day is first processed, its future is unknown, so its index is pushed onto the stack.

A day is removed only when a warmer day appears

An index is popped only when:

```
current_temperature > temperatures[previous_day]
```

Therefore, the current day is valid as a warmer day.

The current day is the first warmer day

If an earlier processed day had been warmer, the index would already have been popped.

Therefore, when it is finally popped, the current day is the nearest warmer day to its right.

---

12. Complexity

Time complexity: O(n)

Although there is a nested `while` loop, the algorithm is not O(n²).

Each index is:

- Pushed once
- Popped at most once

Therefore, total stack operations are at most proportional to `2n`.

```
O(n) pushes + O(n) pops = O(n)
```

Space complexity: O(n)

For a strictly decreasing input:

```
[100, 90, 80, 70, 60]
```

nothing is popped, so the stack contains all indices.

Thus:

- Auxiliary space: **O(n)**
- Output space: **O(n)**

---

13. Edge cases

Empty input

```
daily_temperatures([])
# []
```

One day

```
daily_temperatures([75])
# [0]
```

Strictly increasing

```
daily_temperatures([60, 70, 80, 90])
# [1, 1, 1, 0]
```

Every new day resolves the preceding day.

Strictly decreasing

```
daily_temperatures([90, 80, 70, 60])
# [0, 0, 0, 0]
```

No day has a warmer future day.

Equal temperatures

```
daily_temperatures([70, 70, 71])
# [2, 1, 0]
```

Equal temperatures are not warmer.

That is why the condition must use:

```
current_temperature > previous_temperature
```

not:

```
current_temperature >= previous_temperature
```

Repeated rises and drops

```
daily_temperatures([70, 75, 71, 72, 76])
# [1, 3, 1, 1, 0]
```

The same warmer day may resolve multiple earlier days.

---

14. Common mistakes

Storing temperatures instead of indices

This loses the information needed to calculate distance.

Incorrect:

```
stack.append(current_temperature)
```

Correct:

```
stack.append(current_day)
```

Using only one `if`

A single current temperature may resolve several previous days.

Incorrect:

```
if stack and current_temperature > temperatures[stack[-1]]:
    previous_day = stack.pop()
```

Correct:

```
while stack and current_temperature > temperatures[stack[-1]]:
    previous_day = stack.pop()
```

Using >=

The problem asks for a strictly warmer day.

```
current_temperature > temperatures[stack[-1]]
```

Returning unresolved stack values

Unresolved days should remain zero. No additional processing is required after the loop.

---

15. Stack use in parsing

Stacks are frequently used to parse nested expressions.

Example:

```
3[a2[c]]
```

Expected result:

```
accaccacc
```

A parser can push:

- Previous partial string
- Repetition count
- Opening-scope state

When `]` is encountered, it pops the previous state and combines it with the completed inner expression.

Conceptually:

```
read 3[
push count=3 and previous_string=""
read a
read 2[
push count=2 and previous_string="a"
read c
read ]
build "a" + "c" * 2 = "acc"
```

```
read ]
build "" + "acc" * 3 = "accaccacc"
```

This is a regular stack use case, not necessarily a monotonic stack.

---

16. Stack use for undo and state

A basic undo system stores prior states:

```
class TextEditor:
    def __init__(self) -> None:
        self.text = ""
        self.history: list[str] = []

    def write(self, value: str) -> None:
        self.history.append(self.text)
        self.text += value

    def undo(self) -> None:
        if self.history:
            self.text = self.history.pop()
```

Example:

```
editor = TextEditor()

editor.write("Hello")
editor.write(" World")

print(editor.text)  # Hello World

editor.undo()

print(editor.text)  # Hello
```

For large state objects, storing a complete copy may be expensive. Production systems often store:

- Reversible commands
- Deltas
- Events
- Snapshots at intervals

This connects stack reasoning to:

- Command pattern
- Event sourcing
- Transaction rollback
- Workflow state restoration

---

17. Interview explanation

A strong concise explanation would be:

> The brute-force solution scans forward from every day and takes $O(n^2)$. I can avoid repeated scans using a monotonically decreasing stack of unresolved indices. When a warmer temperature arrives, I repeatedly pop all colder days because the current index is their first warmer future day. Each index is pushed once and popped at most once, so the total time is $O(n)$, with $O(n)$ auxiliary space.

Final answer summary

Area	Key idea
Regular stack	LIFO processing without ordering
Monotonic stack	Maintains increasing or decreasing values
Parsing	Saves nested partial state
Undo/state	Restores the most recent previous state
Daily Temperatures	Next greater element to the right
Brute force	$O(n^2)$ time
Optimized	$O(n)$ time and $O(n)$ space
Stack content	Store indices, not only values
Critical condition	Use strict $>$ for “warmer”

Day 6 — Finance Analytics Service PoC

An interview-grade, locally executable FastAPI service for **budget-versus-actual analysis** and **expense exception triage**. The production profile uses PostgreSQL and Redis; the no-Docker profile uses SQLite and an in-process TTL cache with the same versioned-key contract.

1. Problem statement

Finance teams commonly receive budget data at a cost-centre/month grain and expense transactions from an ERP. Analysts then spend time reconciling totals, finding overspend, reviewing large or unapproved expenses, and preparing drill-down evidence for department owners.

Users

- FP&A analyst: compares budget and actuals, identifies overspend, and explains trends.
- Finance controller: reviews high-risk or unapproved expense exceptions.
- Department owner: drills into the transactions driving a variance.
- Platform engineer: operates ingestion, API, database, cache, logs, and health checks.

Business value

- Reduces manual spreadsheet reconciliation.

- Gives a consistent definition of variance and exception priority.
- Supports faster review through stable pagination and drill-down.
- Makes ingestion retry-safe through idempotency.
- Demonstrates measurable latency improvement from caching.

Scope

- Deterministic synthetic finance data for 2025.
- Departments, cost centres, budgets, vendors, expenses, approval status, and ingestion batches.
- JSON ingestion with validation and idempotency.
- Variance, trend, exception, drill-down, and statistical-analysis APIs.
- PostgreSQL reference DDL and analytical SQL.
- Redis-backed versioned caching with an in-memory development fallback.
- Structured logs, correlation IDs, health checks, metrics, tests, and benchmark scripts.

Non-goals

- General ledger accounting, accruals, FX conversion, tax logic, or multi-entity consolidation.
- Fraud detection or claims that an exception is waste, abuse, or policy violation.
- Production IAM, row-level security, secrets management, distributed tracing backend, or rate limiting.
- Streaming ingestion, CDC, data warehouse orchestration, or a user interface.
- A causal conclusion from the included hypothesis test.

2. Requirements

Functional requirements

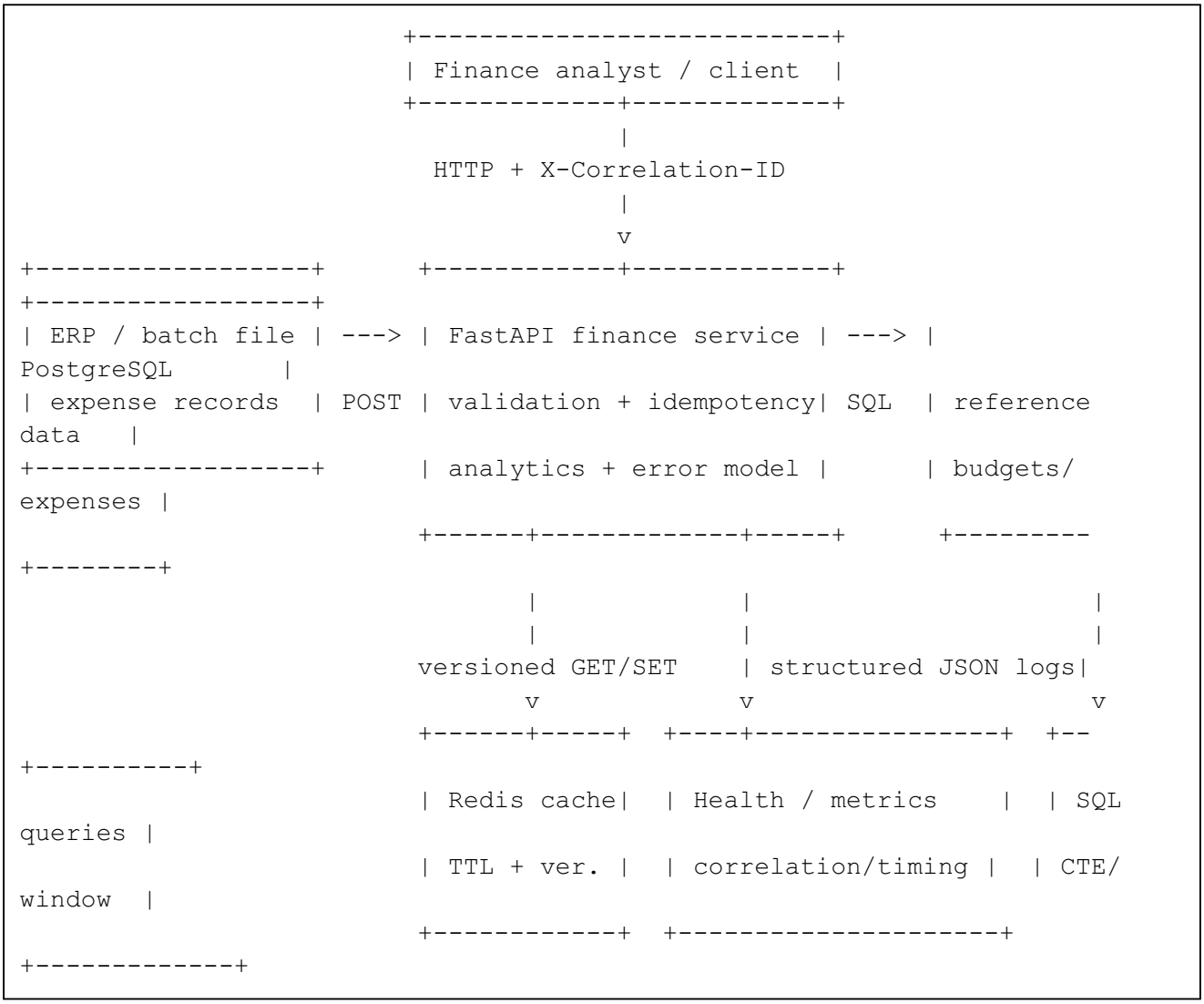
1. Load reference data, budgets, and expense transactions.
2. Reject invalid amounts, statuses, dates, unknown references, missing budgets, and duplicate records inside one payload.
3. Make a retried ingestion request safe with `Idempotency-Key` plus a payload hash.
4. Return budget, actual, variance, and variance percentage by department and month.
5. Return monthly trend data with a rolling three-month actual total.
6. Rank expense exceptions by a documented composite score.
7. Provide stable cursor pagination for exception and drill-down results.
8. Compare approved and non-approved expense amounts with a Welch t-test and 95% confidence interval.
9. Expose liveness, readiness, and lightweight operational metrics.
10. Measure analytical-query latency with and without caching.

Non-functional requirements

- Correctness: aggregate results must reconcile to source totals.
- Reliability: ingestion is transactional and idempotent.
- Performance: common aggregate views are cached with bounded TTL.

- Consistency: deterministic sort order under pagination.
- Observability: JSON logs, correlation IDs, response timing, cache counters, and health checks.
- Security baseline: optional API key, input limits, no request-body logging, parameterized SQL, and generic 500 responses.
- Testability: local SQLite profile and deterministic seed data.
- Portability: Docker Compose for PostgreSQL, Redis, and API.

3. End-to-end architecture



Request flow

1. Middleware accepts or creates a correlation ID and starts a timer.
2. FastAPI validates query, header, and body fields.
3. Ingestion checks the idempotency key and canonical payload hash.
4. The service validates foreign references and budget coverage.
5. One database transaction writes the ingestion batch and new expenses.
6. Successful ingestion increments the cache namespace version.
7. Analytics reads a versioned cache key; a miss executes parameterized analytical SQL and stores JSON with TTL.

8. Middleware returns correlation and response-time headers and writes a structured log.

4. Implementation milestones

Milestone	Deliverable	Verification
1. Domain and data	Problem boundaries, deterministic dataset, validation rules	CSV counts and validation tests
2. Persistence	PostgreSQL schema, constraints, indexes, SQLAlchemy models	Seed script and readiness check
3. Ingestion	Transactional write, idempotency key, payload hash, duplicate protection	Replay and conflict tests
4. Analytics	Variance, exceptions, trends, drill-down	Reconciliation and API tests
5. Cache	Redis/in-memory implementation, TTL, version invalidation	HIT/MISS and invalidation tests
6. Statistics	Welch test, 95% CI, effect size, limitations	Statistical endpoint test
7. Production baseline	Logs, correlation IDs, error envelope, health, metrics, optional API key	Health/security tests
8. Evaluation	Quality, latency, operational/business metrics	`METRICS.md`
9. Interview readiness	Demo scripts, design narrative, likely questions	`docs/demo_script.md`

5. Pseudocode before code

Idempotent ingestion

```
function ingest(idempotency_key, payload):
    validate payload shape, dates, status, amount, and batch uniqueness
    payload_hash = sha256(canonical_sorted_payload)

    existing_batch = find batch by idempotency_key
    if existing_batch exists:
        if existing_batch.payload_hash differs:
            return 409 conflict
        return original result with replayed=true

    resolve cost-centre and vendor IDs
    reject unknown references or missing monthly budgets
    find source records already stored

    begin transaction
        insert ingestion batch
        insert only unseen source records
    commit

    increment analytics cache version
    return inserted count and batch ID
```

Cached variance query

```
function variance(filters):
    version = cache.get("analytics-version")
    key = stable_json(version + filters)

    if cache contains key:
        increment cache-hit metric
        return cached data and HIT

    increment cache-miss metric
    rows = execute budget CTE + actual CTE + left join
    cache rows with TTL
    return rows and MISS
```

Stable cursor pagination

```
sort by exception_score descending, expense_id ascending
cursor contains last_score and last_expense_id
next query keeps rows where:
    score < last_score
    OR score = last_score AND expense_id > last_expense_id
fetch limit + 1 to decide whether another page exists
```

6. Data and schema design

Dataset

The generated dataset is deterministic:

- 8 departments
- 24 cost centres
- 30 vendors
- 288 monthly budgets
- 2,330 expense transactions
- Periods: January–December 2025

Files are under `data/`; regenerate them with:

```
python scripts/generate_seed.py
```

Core entities

Entity	Grain	Important constraints
Department	one department	unique code
Cost centre	one cost centre	unique code, department FK
Vendor	one vendor	unique code, risk tier check
Budget	cost centre + month	unique grain, non-negative amount

Expense	source system + source record	positive amount, valid status, reference FKs
Ingestion batch	idempotency key	unique key, payload hash, row counts

Index choices

- `budgets(period, cost_centre_id)` supports period filtering and budget joins.
- `expenses(period, cost_centre_id)` supports variance, trend, and drill-down.
- `expenses(vendor_id, period)` supports vendor analysis.
- `expenses(approval_status, period)` supports approval review.

For a much larger table, consider monthly range partitioning, covering indexes based on real query plans, and pre-aggregated monthly facts.

7. API surface

Method	Endpoint	Purpose
POST	`/v1/ingestion/expenses`	Validated, idempotent expense ingestion
GET	`/v1/analytics/variance`	Department/month budget-versus-actual
GET	`/v1/analytics/trends`	Monthly trend and rolling three-month actual
GET	`/v1/analytics/exceptions`	Ranked, cursor-paginated exception candidates
GET	`/v1/analytics/drilldown`	Transaction detail for cost centre/month
GET	`/v1/analytics/statistics/approval-amount-test`	Welch test, CI, and limitations
GET	`/health/live`	Process liveness
GET	`/health/ready`	Database and cache readiness
GET	`/internal/metrics`	In-process request/cache metrics

OpenAPI documentation is available at `/docs` after startup.

Detailed curl examples are in `docs/api_examples.md`.

8. Local setup

Option A: no Docker

This path uses SQLite and the in-memory TTL cache.

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"

export DATABASE_URL=sqlite:///./finance.db
export REDIS_URL=memory://
export AUTO_CREATE_SCHEMA=true
```

```
python scripts/seed_db.py
uvicorn app.main:app --reload
```

Test:

```
curl -i http://localhost:8000/v1/analytics/variance
```

Option B: PostgreSQL and Redis with Docker Compose

```
docker compose up --build -d

docker compose exec api python scripts/seed_db.py
curl -i http://localhost:8000/v1/analytics/variance
```

Stop and remove data:

```
docker compose down -v
```

Run tests

```
pytest
```

Run measured local benchmark

```
export DATABASE_URL=sqlite:///./finance.db
export REDIS_URL=memory://
python scripts/benchmark.py --iterations 300
```

Run full-stack HTTP benchmark

```
python scripts/http_benchmark.py --base-url http://localhost:8000 --
iterations 300
```

9. Exception score

The PoC score is transparent and deterministic:

```
expense as % of monthly cost-centre budget
+ 50 for REJECTED or 30 for PENDING
+ 25 for HIGH-risk vendor or 10 for MEDIUM-risk vendor
```

This is a **review-priority score**, not a probability and not a fraud label. A production version should be calibrated with historical review outcomes, false-positive cost, policy severity, vendor context, and explainability requirements.

10. Statistical calculation

The endpoint compares approved expense amounts with pending/rejected expense amounts using Welch's two-sample t-test because the group variances and sample sizes may differ. It returns:

- group sample sizes and means;
- non-approved minus approved mean difference;
- 95% confidence interval for the difference;
- t statistic and p-value;
- Cohen's d effect size;
- explicit limitations.

The result is not causal. The synthetic generator intentionally makes large expenses somewhat more likely to remain pending or be rejected, observations are clustered, and amounts are heavy-tailed.

11. Error handling

Errors use a consistent envelope:

```
{
  "error": {
    "code": "validation_error",
    "message": "request validation failed",
    "details": [],
    "correlation_id": "..."
  }
}
```

- 400: invalid business rule or cursor.
- 401: missing/incorrect API key when configured.
- 404: missing cost centre.
- 409: idempotency-key conflict or concurrent duplicate.
- 422: request schema validation failure.
- 500: generic message; detailed exception stays in logs.

12. Security baseline

Included:

- Optional `X-API-Key` protection through `API_KEY`.
- Parameterized SQL.
- Request size bounded to 5,000 rows per ingestion batch.
- Strict status and date validation.
- No raw request-body logging.
- Generic internal-error response.
- Database constraints as a second validation layer.

Production additions:

- OAuth/OIDC service and user authentication.
- Role/department authorization and PostgreSQL row-level security.
- TLS, secret manager, key rotation, network policies, WAF/rate limits.
- Audit log for data access and configuration changes.
- PII classification, retention, deletion, and masking policies.

13. Observability

- JSON logs include timestamp, level, logger, message, and correlation ID.
- Request completion logs include method, path, status, and duration.
- Responses include `X-Correlation-ID`, `X-Response-Time-Ms`, and analytics cache status.
- Readiness checks database and cache connectivity.
- Metrics track request count, 5xx count, average latency, status counts, cache hits, cache misses, and hit ratio.

Production additions would export OpenTelemetry traces and Prometheus metrics rather than relying on process-local counters.

14. Evaluation and measured metrics

See `METRICS.md` for executed results. The included local run reports:

- quality: budget and actual reconciliation absolute error;
- latency: direct analytical SQL versus warm cache p50/p95;
- operational metric: cache hit ratio;
- business triage metric: count and spend share of exception candidates;
- statistical result: p-value and confidence interval.

All local results are labeled with their environment. PostgreSQL/Redis production numbers are deliberately left for an executed full-stack benchmark.

15. Repository structure

```
.
├── app/
│   ├── api/routes/          # HTTP endpoints
│   ├── repositories/       # SQL and persistence access
│   ├── services/           # ingestion, cache use, statistics
│   ├── cache.py             # Redis and in-memory TTL cache
│   ├── config.py            # environment settings
│   ├── db.py                # engine/session lifecycle
│   ├── logging.py           # structured JSON logs
│   ├── metrics.py           # lightweight counters
│   ├── middleware.py        # correlation ID and timing
│   ├── models.py            # SQLAlchemy schema
│   ├── schemas.py           # Pydantic contracts
│   └── main.py              # app factory and handlers
```

```
├── data/                       # deterministic CSV seed data
├── docs/                       # API, demos, interview notes
├── scripts/                    # generation, seeding, stats, benchmarks
├── sql/                        # PostgreSQL DDL and analytical SQL
├── tests/                      # API, ingestion, pagination, security tests
├── docker-compose.yml
├── Dockerfile
├── METRICS.md
└── README.md
```

16. Trade-offs

Decision	Benefit	Cost / alternative
Relational normalized schema	Constraints and explainable joins	Warehouse/star schema may be better for very large analytics
Raw analytical SQL behind repository	Clear CTE/window-function reasoning	More dialect-specific than pure ORM
Versioned cache keys	Safe invalidation without key scans	Old keys remain until TTL
Synchronous SQLAlchemy	Simple interview PoC and predictable transactions	Async may help under high I/O concurrency but adds complexity
Cursor pagination	Stable under inserts and large offsets	Cursor is opaque and tied to sort order
Transparent rule score	Explainable and easy to validate	Not calibrated to review outcomes
SQLite local profile	Runs without Docker	Does not reproduce PostgreSQL optimizer or Redis network latency

17. Known limitations and next steps

1. Replace API key with OAuth/OIDC and fine-grained authorization.
2. Add Alembic migrations rather than automatic table creation.
3. Run PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` and tune indexes from real plans.
4. Add Redis failure policy, timeouts, circuit breaker, and cache stampede protection.
5. Export metrics/traces to Prometheus and OpenTelemetry backends.
6. Add duplicate-invoice and policy-rule endpoints.
7. Add fiscal calendars, currencies, accruals, forecasts, and organizational hierarchies.
8. Add snapshot or aggregate tables for high-volume workloads.
9. Validate exception rules against labeled reviewer outcomes and measure precision/recall plus review-time reduction.
10. Add concurrency/load tests and PostgreSQL/Redis integration tests in CI.

18. Demo and interview material

- `docs/demo_script.md`: three-minute business demo, two-minute design explanation, and extended technical walkthrough.

- `docs/interview_questions.md`: likely interviewer questions and answer anchors.
- `docs/api_examples.md`: runnable API examples.
- `DAY6_MENTOR_GUIDE.md`: interview-focused learning notes and explanation.

Day 6 Mentor Guide — Finance Analytics Integration PoC

Beginner-friendly summary

This project combines the first five days into one system:

- **Python/backend:** a FastAPI application with validation, errors, logs, tests, and configuration.
- **API contracts:** typed request/response models, status codes, idempotency, correlation IDs, and stable pagination.
- **SQL/data:** a relational finance schema, constraints, indexes, CTEs, joins, aggregation, and window functions.
- **Concurrency/resilience awareness:** bounded request inputs, transactional writes, cache failure boundaries, and health checks.
- **Statistics:** a confidence interval, Welch hypothesis test, effect size, and limitations.

The important interview lesson is not “I built four endpoints.” It is: **I defined a financial grain, protected correctness at multiple layers, made retries safe, exposed explainable analytics, measured the system, and stated what the PoC cannot prove.**

---

1. Problem statement, users, business value, scope, and non-goals

Problem statement

A finance organization has monthly budget allocations at department and cost-centre level and expense transactions arriving from an ERP. Analysts need to answer:

1. Where are actual expenses above or below budget?
2. Is the variance a one-month spike or a sustained trend?
3. Which individual expenses deserve review first?
4. Can the analyst drill from an aggregate to source transactions?
5. Can a retried ingestion request avoid duplicates?
6. Can the service return consistent results quickly enough for repeated dashboard use?

Users

- **FP&A analyst:** monitors variance and prepares commentary.
- **Controller:** reviews unapproved, rejected, unusually large, or risky-vendor expenses.
- **Department owner:** sees the transactions behind a cost-centre variance.
- **Data/platform engineer:** operates ingestion, storage, cache, API, logs, and tests.

Business value

- Consistent and reproducible variance definitions.
- Lower manual reconciliation effort.

- Faster exception review through ranking and drill-down.
- Reduced duplicate-record risk from idempotent ingestion.
- Faster repeated reads through cache-aside serving.
- Better auditability through correlation IDs and source-record lineage.

Scope

- One legal entity and one currency.
- Monthly budgets and transaction-level expenses.
- Deterministic synthetic data for January–December 2025.
- Finance analytics APIs and one statistical comparison.
- PostgreSQL and Redis production profile.
- SQLite and in-memory cache local profile.

Non-goals

- General ledger posting, double-entry accounting, accruals, FX, tax, or consolidation.
- Forecasting or ML-based anomaly detection.
- Fraud determination.
- Production-grade identity, fine-grained authorization, data retention, or UI.
- Causal inference from approval status.

---

2. Functional and non-functional requirements

Functional requirements

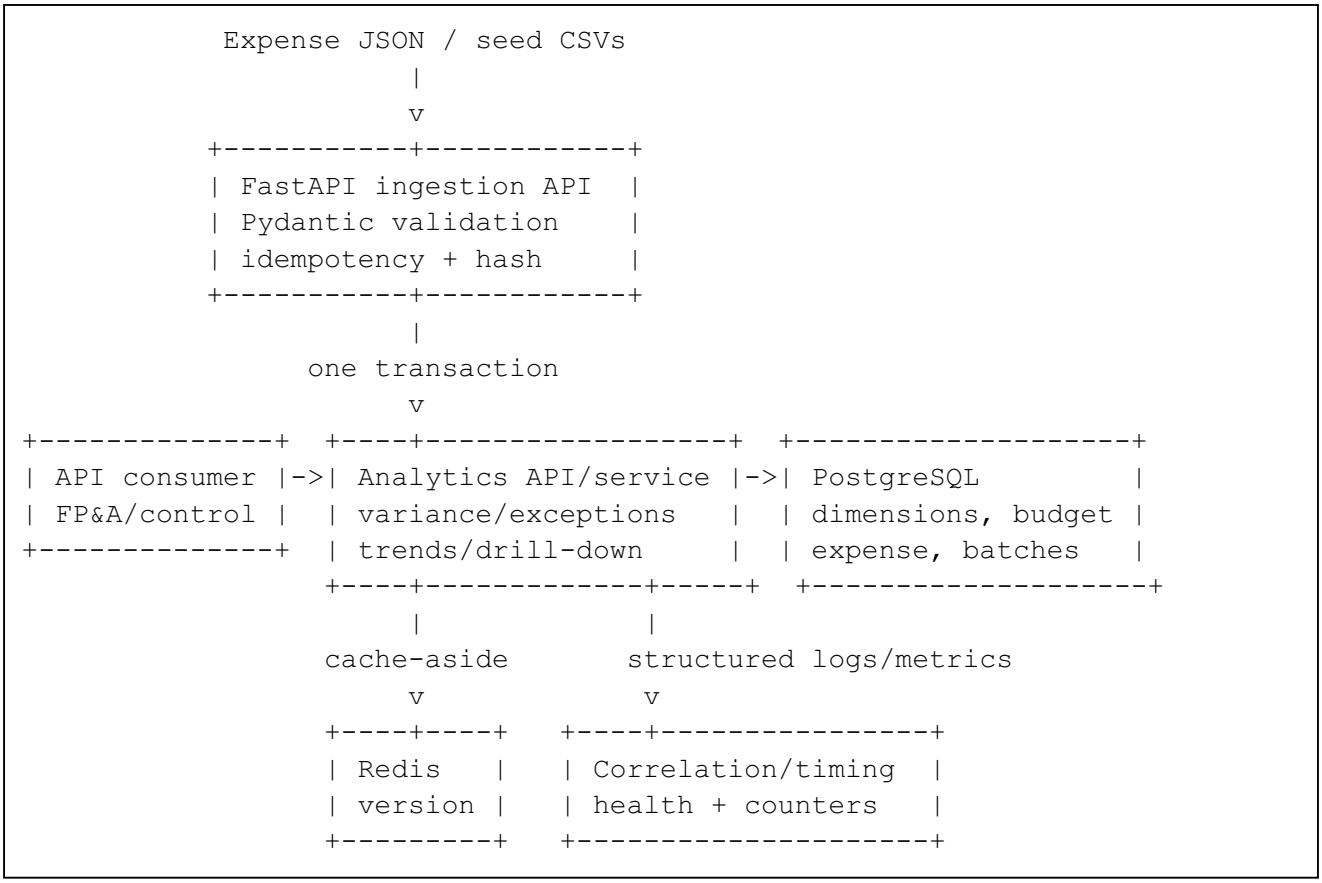
ID	Requirement	Acceptance condition
F1	Seed realistic finance data	CSVs contain all required entities and deterministic values
F2	Validate ingestion	Bad amount, status, date, reference, duplicate payload row, or missing budget is rejected
F3	Idempotent write	Same key/same payload replays; same key/different payload conflicts
F4	Variance summary	Returns budget, actual, variance, and variance percent by department/month
F5	Exception ranking	Returns documented score and explainable reasons
F6	Trend view	Returns monthly values and rolling three-month actual
F7	Drill-down	Returns source transactions for cost centre/month
F8	Stable pagination	No overlap between consecutive pages under unchanged data
F9	Statistical result	Returns CI, test statistic, p-value, effect size, and limitations
F10	Operability	Liveness, readiness, logs, correlation IDs, and lightweight metrics are present

Non-functional requirements

Attribute	Design response
Correctness	database constraints, transactional writes, reconciliation metric, tests
Reliability	idempotency key, payload hash, source-record uniqueness
Performance	indexed filters, aggregated SQL, cache-aside, measured cold/warm latency
Scalability	cursor pagination and a documented aggregate/partitioning path
Security	optional API key, parameterized SQL, bounded payloads, generic 500 responses
Observability	JSON logs, timing headers, correlation IDs, health checks, counters
Maintainability	app factory, repository/service/API layers, deterministic fixtures
Explainability	transparent exception score and explicit statistical limitations

---

3. End-to-end architecture



Correctness boundaries

1. **Pydantic** protects the API contract.
2. **Service validation** checks reference and budget existence.
3. **Database constraints** protect stored state even if another writer bypasses the API.
4. **Transaction** makes batch metadata and expense writes atomic.
5. **Reconciliation** checks aggregate output against source totals.
6. **Tests** protect retry, cache, pagination, statistics, and error behavior.

---

4. Implementation milestones

Milestone 1 — Model the financial grain

Before coding endpoints, state the grain:

- Budget: one row per **cost centre + month**.
- Expense: one row per **source system + source record ID**.
- Variance: aggregate expenses to the budget grain before comparison.

A common interview mistake is to begin with API routes without defining grain. That creates duplicate joins, incorrect totals, and ambiguous ownership.

Milestone 2 — Build deterministic data

Use a fixed random seed. Include normal variation and deliberate review cases:

- quarter-end seasonality;
- occasional cost-centre overspend;
- amount-dependent pending/rejected status;
- vendor risk tiers;
- a few very large transactions.

Determinism matters because test and benchmark results must be repeatable.

Milestone 3 — Add relational constraints and indexes

Use unique constraints for business grains and source identity. Index the actual filters and joins rather than adding indexes to every field.

Milestone 4 — Implement idempotent ingestion

Separate two concepts:

- **Request idempotency:** same logical request is safe to retry.
- **Record deduplication:** the same source record is not inserted by another request.

This project uses both.

Milestone 5 — Implement analytical SQL

- Variance: budget CTE + actual CTE + left join.
- Trend: monthly aggregate + window frame.
- Exceptions: joined context + composite score.
- Drill-down: source detail with deterministic ordering.

Milestone 6 — Add cache-aside serving

Use normalized filters and a version in every key. On ingestion, increment the version. This avoids a wildcard delete.

Milestone 7 — Add statistics honestly

Define the comparison, calculate the interval and test, and list reasons not to interpret it causally.

Milestone 8 — Verify and measure

Run tests, reconciliation, query benchmarks, cache hit ratio, and a business triage metric. Label the environment.

---

5. Pseudocode

Ingestion

```
INPUT: idempotency key, list of expenses

validate request fields and cross-field dates
canonicalize rows and calculate SHA-256 hash

lookup ingestion batch by idempotency key
if found:
    if stored hash != current hash: return conflict
    return original batch as replay

resolve cost centres and vendors
check every cost-centre/month has a budget
find records already stored by source identity

begin transaction
    insert ingestion batch
    insert unseen expense rows linked to batch
commit

increment cache namespace version
return batch ID, received count, inserted count
```

Variance

```
budgeted = group budget by department and month
actuals = group expense amount by department and month
left join actuals onto budgeted
variance = actual - budget
variance_pct = variance / budget * 100, unless budget is zero
```

Exception pagination

```
score = budget share + approval penalty + vendor-risk penalty
sort by score DESC, expense_id ASC
cursor = last score + last expense_id
next page filters after that tuple and fetches limit + 1
```

Welch confidence interval

```
split amounts into approved and non-approved groups
calculate each mean, variance, and sample size
standard_error = sqrt(var1/n1 + var2/n2)
degrees_of_freedom = Welch-Satterthwaite formula
```

```
critical_value = t quantile at 97.5%
CI = mean_difference +/- critical_value * standard_error
run two-sided Welch t-test
calculate pooled-standard-deviation effect size separately
```

---

6. Key implementation decisions

Why actuals are derived from expense rows

A separate actual table could drift from transaction detail. In this PoC, actual is the sum of stored expenses, so drill-down and aggregate share one source of truth. At larger scale, a maintained aggregate fact may be introduced, but it must reconcile back to transactions.

Why budget is the left side of the variance join

The finance question is “How did we perform against an allocated budget?” A budget period with zero expenses should still appear. An inner join would silently remove it.

Why payloads are sorted before hashing

Two JSON arrays with the same logical records in different orders should not become different requests. Sorting by source identity before hashing makes the idempotency definition semantic rather than byte-order dependent.

Why successful ingestion increments a cache version

Deleting keys by pattern is operationally risky. Versioning makes old entries unreachable immediately after the increment and lets TTL reclaim them later.

Why the score is not ML

There is no labeled reviewer outcome. A transparent rule is a defensible baseline. An interviewer should hear that ML is not automatically better; it needs labels, an objective, costs, evaluation, drift monitoring, and governance.

Why Welch’s test

The approved and non-approved groups need not have equal variance or sample size. Welch’s method relaxes the equal-variance assumption. It still relies on independent observations and mean-based inference, which are imperfect here.

---

7. Data and schema design

Dimensions and facts

- `departments`: business ownership.
- `cost_centres`: lower-level budget and accountability unit.
- `vendors`: counterparty and risk tier.
- `budgets`: monthly allocation fact.
- `expenses`: transaction fact and actual source.
- `ingestion_batches`: operational lineage and idempotency record.

Important constraints

- department, cost-centre, and vendor codes are unique;
- one budget exists per cost-centre/month;
- amount is positive for expenses and non-negative for budgets;
- approval status is from a controlled set;
- source-system/source-record is unique;
- idempotency key is unique;
- every expense points to valid reference data and an ingestion batch.

Query patterns and indexes

The main access path is period + cost centre, so composite indexes lead with period. Vendor and approval indexes support exception slices. Do not claim an index is optimal until a PostgreSQL query plan is measured.

---

8. API behavior

Variance endpoint

Filters: period range and optional department. Response models filter and validate output. Cache status is visible in `X-Cache`.

Exception endpoint

Filters: period range, optional department, minimum amount, minimum score, page limit, and cursor. Each result includes reasons derived from its components.

Trend endpoint

Returns budget, actual, variance, and rolling three-month actual. The first two months naturally contain one- and two-month partial windows.

Drill-down endpoint

Uses cost-centre code and month. Stable cursor order is transaction date descending and unique expense ID ascending.

Ingestion endpoint

Requires `Idempotency-Key`. Same request can be replayed safely. Different payload under the same key returns 409.

---

9. Tests

The repository executes 13 tests covering:

- first cache miss and later hit;
- aggregate shape and reconciliation behavior;
- rolling-window calculation;
- real statistical output;
- invalid period range error envelope;

- exception pagination without overlap;
- invalid cursor rejection;
- drill-down pagination;
- idempotent replay;
- key reuse conflict;
- cross-period date validation;
- cache invalidation after ingestion;
- liveness, readiness, metrics, and optional API key.

What is not yet tested

- actual PostgreSQL/Redis integration in this execution environment;
- concurrent same-key requests;
- Redis outage behavior;
- load, soak, and connection-pool saturation;
- migration rollback;
- authorization boundaries.

---

10. Error handling

Use a stable machine-readable code, human-readable message, optional details, and correlation ID. Do not expose stack traces or SQL errors to clients.

Important mapping:

- 400 for domain validation and invalid cursor;
- 401 for API-key failure;
- 404 for unknown drill-down resource;
- 409 for idempotency conflict;
- 422 for request contract failure;
- 500 for unexpected failures.

The database transaction rolls back on integrity failure.

---

11. Security

Included baseline

- optional API key;
- parameterized SQL;
- bounded batch size;
- input validation;
- generic internal errors;
- no payload logging;
- database constraints.

Production controls

- OAuth/OIDC and service identities;
- role and department authorization;
- row-level security or centrally enforced query scope;
- TLS and secret manager;
- audit access logs;
- retention, deletion, encryption, and data classification;
- rate limiting and abuse controls.

---

12. Observability

Logs

Every request gets a correlation ID. Completion logs include method, path, status, and latency. Ingestion logs batch ID and row count. Logs are JSON so a log backend can parse them.

Health

- Liveness answers whether the process can respond.
- Readiness checks database and cache connectivity.

Metrics

The PoC tracks request count, status count, 5xx count, average request latency, cache hits, cache misses, and hit ratio. Production should use histogram buckets and external aggregation.

---

13. Evaluation

Quality

Reconciliation absolute error compares the sum returned by analytical queries with source table totals. The measured deterministic run produced zero error for budget and actual.

Latency

The local benchmark separately measures direct analytical SQL and warm versioned cache. It reports p50 and p95, not only an average.

Operational metric

Cache hit ratio shows whether the cache is serving repeated analytics effectively.

Business metric

Exception spend share measures how much spend is routed into review by the current threshold. It is not “money saved” and should not be presented as such.

Statistical metric

The project reports p-value, confidence interval, and effect size. Business action should consider all three plus the limitations.

---

14. Measured results

The executed local environment contained 2,330 expenses. Results are in `METRICS.md`.

Key measured outcomes:

- 13 tests passed;
- budget reconciliation absolute error: 0.00;
- actual reconciliation absolute error: 0.00;
- direct SQL p50: 1.5992 ms;
- warm cache p50: 0.0778 ms;
- measured p50 speed-up: 20.55x;
- cache hit ratio: 99.67% after one priming miss;
- exception candidates: 834;
- exception spend share: 56.52%;
- Welch mean-difference 95% CI: 4,836.35 to 8,851.39 in synthetic currency units.

Do not use these numbers as PostgreSQL/Redis or production claims. The benchmark was SQLite plus an in-process cache on the stated machine.

---

15. Repository and README outline

The repository separates transport, services, repositories, storage contracts, scripts, SQL, data, tests, and interview documentation. The README follows this order:

1. problem and boundaries;
2. requirements;
3. architecture;
4. milestones and pseudocode;
5. schema and APIs;
6. setup and execution;
7. statistics, errors, security, and observability;
8. measured evaluation;
9. repository structure and trade-offs;
10. limitations, next steps, and demo references.

This ordering is interview-friendly because it explains **why**, then **what**, then **how**, then **evidence**.

---

16. Demo preparation

Three-minute business demo

1. State the finance reconciliation and review problem.
2. Show engineering variance and trend.
3. Show top exception reasons and drill-down.
4. Show miss/hit cache headers.
5. Replay an ingestion request safely.
6. Show the statistical endpoint and state non-causality.

Two-minute design explanation

1. State budget and expense grains.
2. Explain transactional idempotency and source uniqueness.
3. Explain versioned cache invalidation.
4. Explain stable cursor tuple.
5. Explain logs, health, metrics, and production evolution.

Five-to-ten-minute technical depth

Open the models, ingestion service, SQL repository, cache interface, statistical service, tests, and measured metrics in that order.

---

17. Limitations and next steps

Limitations

- synthetic data and designed patterns;
- one currency/entity and monthly calendar;
- transparent heuristic rather than calibrated risk;
- process-local metrics;
- no migration tool;
- no full-stack benchmark in this environment;
- cache invalidation has a small post-commit failure window;
- no snapshot isolation across paginated requests;
- no user/department authorization;
- no warehouse-scale strategy implemented.

Next steps

1. Add Alembic migrations.
2. Execute PostgreSQL and Redis integration tests in CI.
3. Add query-plan capture and index tuning.
4. Add OpenTelemetry and Prometheus.
5. Add cache timeouts, fail-open policy, circuit breaker, and stampede control.
6. Add OAuth/OIDC, department authorization, and audit logs.
7. Add duplicate-invoice and policy-rule analysis.

8. Add reviewer labels and evaluate precision at K, false-positive burden, and review-time reduction.
9. Add partitions or aggregate tables only after workload measurement.
10. Add concurrency and load tests.

---

18. Likely interviewer questions

1. Why PostgreSQL rather than NoSQL?
2. Why derive actuals from expense detail?
3. Why is the budget side a left join?
4. How does idempotency differ from deduplication?
5. What happens under two concurrent requests with the same key?
6. Why use a payload hash?
7. Why version cache keys rather than delete by pattern?
8. Where can database/cache inconsistency occur?
9. Why cursor pagination, and what changes can still cause movement?
10. Why is UUID a tie-breaker?
11. How would you evaluate the exception score?
12. Why is the hypothesis test not causal?
13. What assumptions does Welch's test retain?
14. What would change at 100 million rows?
15. What do zero reconciliation error and passing tests not prove?
16. Which production control would you add first?
17. Would async SQLAlchemy improve this service?
18. How would you protect department-specific financial data?
19. How would you handle Redis failure?
20. How would you explain the measured metrics without overstating them?

Use `docs/interview_questions.md` for answer anchors.

API Examples

Variance summary

```
curl -i \
  -H 'X-Correlation-ID: demo-variance-001' \
  'http://localhost:8000/v1/analytics/variance?
period_from=2025-01-01&period_to=2025-12-01&department_code=ENG'
```

Run twice and compare `X-Cache: MISS` with `X-Cache: HIT`.

Trend view

```
curl -s \
  'http://localhost:8000/v1/analytics/trends?
period_from=2025-01-01&period_to=2025-12-01&department_code=MKT'
```

Top exceptions

```
curl -s \
  'http://localhost:8000/v1/analytics/exceptions?
period_from=2025-01-01&period_to=2025-12-01&min_amount=5000&min_score=35&lim
it=5'
```

Use the returned `next_cursor` in the next request:

```
curl -sG 'http://localhost:8000/v1/analytics/exceptions' \
  --data-urlencode 'limit=5' \
  --data-urlencode 'cursor=<NEXT_CURSOR>'
```

Cost-centre drill-down

```
curl -sG 'http://localhost:8000/v1/analytics/drilldown' \
  --data-urlencode 'cost_centre_code=ENG-01' \
  --data-urlencode 'period=2025-01-01' \
  --data-urlencode 'limit=10'
```

Statistical test

```
curl -s 'http://localhost:8000/v1/analytics/statistics/approval-amount-test'
```

Idempotent ingestion

```
curl -i -X POST 'http://localhost:8000/v1/ingestion/expenses' \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: manual-demo-001' \
  -d '{
    "rows": [
      {
        "source_system": "MANUAL",
        "source_record_id": "MANUAL-DEMO-001",
        "cost_centre_code": "FIN-01",
        "vendor_code": "V001",
        "period": "2025-01-01",
        "transaction_date": "2025-01-20",
        "invoice_number": "INV-MANUAL-DEMO-001",
        "amount": "2500.00",
        "approval_status": "APPROVED",
        "description": "Manual interview demonstration record"
      }
```

```
    ]
  }'
```

Repeat the same request and observe `replayed: true`. Change the amount while retaining the same idempotency key and observe HTTP 409.

Health and metrics

```
curl -s http://localhost:8000/health/live
curl -s http://localhost:8000/health/ready
curl -s http://localhost:8000/internal/metrics
```

Interview Demo Scripts

Three-minute business demo

0:00–0:30 — Problem and user

“Finance teams receive monthly budgets and thousands of ERP expenses. The hard part is not calculating one variance; it is consistently reconciling totals, prioritizing review, and giving department owners a defensible drill-down. This PoC serves FP&A analysts, controllers, and cost-centre owners.”

0:30–1:20 — Variance and trend

1. Call `/v1/analytics/variance?department_code=ENG`.
2. Show monthly budget, actual, absolute variance, and percentage variance.
3. Call `/v1/analytics/trends?department_code=ENG`.
4. Point out the rolling three-month actual, which helps distinguish one-month spikes from sustained pressure.

Say: “The aggregate is produced from real seeded transactions and reconciles to source totals with zero absolute error in the measured run.”

1:20–2:10 — Exception triage and drill-down

1. Call `/v1/analytics/exceptions?limit=5`.
2. Explain one item’s reasons: pending approval, high-risk vendor, or a large share of monthly budget.
3. Use `/v1/analytics/drilldown` for the item’s cost centre and period.

Say: “The score prioritizes human review. It is intentionally transparent and is not presented as fraud probability.”

2:10–2:40 — Reliability and performance

1. Call variance twice.
2. Show `X-Cache: MISS`, then `X-Cache: HIT`.
3. Mention the measured local p50 improvement and qualify the environment.
4. Ingest one new expense twice with the same idempotency key and show the replay result.

2:40–3:00 — Statistical reasoning and close

Call `/v1/analytics/statistics/approval-amount-test`.

Say: “The service reports a Welch confidence interval and p-value, but the README explicitly explains that this synthetic association is not causal. The PoC combines data contracts, SQL, statistics, APIs, caching, tests, and operational controls.”

Two-minute design explanation

0:00–0:35 — Data model and correctness

“Budgets are stored at cost-centre/month grain; expenses are source-system/source-record grain. Database uniqueness and check constraints backstop Pydantic validation. Analytical queries aggregate expenses into actuals and left-join them to budget so a month with no expense still appears.”

0:35–1:05 — Ingestion and consistency

“Ingestion uses an idempotency key plus a SHA-256 hash of a canonical sorted payload. A same-key same-payload retry returns the original batch; same-key different-payload returns 409. Batch and expense writes commit in one transaction.”

1:05–1:30 — Cache and pagination

“Analytics cache keys include a namespace version and normalized filters. Successful ingestion increments the version, so no wildcard key scan is required. Exception pagination sorts by score descending and UUID ascending; the cursor stores both values to avoid offset drift.”

1:30–2:00 — Operations and scale path

“Middleware creates a correlation ID, measures latency, and emits structured logs. Readiness checks database and cache. For scale, I would add Alembic, OpenTelemetry, Redis timeouts and stampede control, PostgreSQL plan analysis, partitioning or aggregates, OAuth/RLS, and load tests.”

Extended 5–10 minute technical walkthrough

1. Contracts and validation

- Show `ExpenseIn` and the cross-field transaction-period validator.
- Explain the 5,000-row bound and reference-data validation.
- Explain why database constraints remain necessary after API validation.

2. Analytical SQL

- Show the variance CTEs and left join.
- Show the trend window frame: `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.
- Show the transparent exception score and deterministic tie-breaker.

3. Idempotency

- Canonicalize and sort records before hashing so payload order does not create a new semantic request.
- Explain replay versus conflict behavior.
- Explain the residual concurrent-request race and unique constraint fallback.

4. Cache strategy

- Read version, normalize filters, build cache key.
- Miss executes SQL and stores JSON with TTL.
- Write increments version.
- Discuss stale-read window, cache-aside behavior, and why versioning avoids expensive key scans.

5. Statistical analysis

- State null and alternative hypotheses.
- Explain why Welch's test is used instead of assuming equal variance.
- Interpret CI, effect size, and p-value separately.
- State non-causality, clustering, heavy-tail, synthetic-data, and multiple-testing limitations.

6. Verification

- Run `pytest` and show 13 passing tests.
- Open `METRICS.md` and explain zero reconciliation error.
- Qualify the SQLite/in-memory benchmark and explain how to collect PostgreSQL/Redis numbers.

7. Production evolution

- Schema migrations, authentication, authorization, RLS, secret management.
- OpenTelemetry and Prometheus.
- Partitioning/materialized aggregates after observing plans and workload.
- Queue-based ingestion for very large batches.
- Reviewer labels and precision/recall for exception quality.

Likely Interviewer Questions and Answer Anchors

Why PostgreSQL rather than a document database?

The domain has stable relationships, strict grains, transactional ingestion, uniqueness constraints, and join-heavy analytics. PostgreSQL supports those directly. A document store might fit raw source payload retention, but it would not replace the relational analytical model here.

Why synchronous endpoints?

The PoC prioritizes clear transaction boundaries and interview readability. FastAPI can execute synchronous route functions without blocking the event loop directly. For a measured high-concurrency workload, I would compare a larger sync connection pool

against SQLAlchemy async with an async driver rather than assuming async is automatically faster.

How is ingestion truly idempotent?

The idempotency key identifies the logical request, and the canonical payload hash detects accidental key reuse with different content. A unique constraint handles concurrent races. Source-system/source-record uniqueness also prevents duplicate business records across separate ingestion batches.

Why not delete all Redis keys after ingestion?

Wildcard deletion is slow and risky. Versioned keys make invalidation an O(1) increment. Old entries become unreachable and expire by TTL. The trade-off is temporary unused memory.

Can cache and database become inconsistent?

Yes. The database commit occurs before cache-version increment, so a process failure in that small gap could leave old keys addressable until TTL. Production options include an outbox/event, a very short TTL, or a transactionally coordinated invalidation mechanism depending on consistency requirements.

Is cursor pagination stable when new data arrives?

It is more stable than offset pagination because it resumes from the last sort tuple. New higher-ranked records can appear before the current position without duplicating previously seen rows. Updates to a previously returned item's score can still move it; snapshot semantics would require a read timestamp or snapshot/version boundary.

Why is expense ID part of the cursor?

Scores are not unique. A deterministic unique tie-breaker prevents skipping or repeating rows with equal score.

What does zero reconciliation error prove?

It proves the tested aggregate query matches source totals for the seeded dataset. It does not prove all business definitions are correct, all filters are tested, or production ingestion is complete.

Is the p-value enough to act on?

No. I would examine effect size, confidence interval, data-generating process, independence, distribution shape, multiple tests, and business cost. This dataset is synthetic and intentionally creates an association.

How would you evaluate exception quality?

Collect reviewer outcomes, define the positive class and review cost, measure precision at K, recall, false-positive burden, calibration if a probability model is introduced, review-time reduction, and financial value recovered. Segment metrics by department, vendor, amount band, and period.

What breaks at 100 million expenses?

Full scans and on-demand aggregation become expensive. I would inspect query plans, partition by period, maintain monthly aggregate facts, use incremental refresh or streaming aggregates, add covering indexes where justified, and separate operational ingestion from analytical serving if workload isolation is needed.

How would you secure department-specific data?

Use authenticated identities, authorization claims, service-to-service credentials, PostgreSQL row-level security or enforced repository filters, audit access, and test for horizontal privilege escalation. The optional API key in the PoC is only a baseline.

How would you make the cache resilient?

Set connection and command timeouts, fail open for reads when acceptable, add circuit breaking, protect against stampedes with request coalescing or locks, monitor hit ratio and evictions, and cap payload size.

Why use a transparent rule score instead of ML?

There is no labeled outcome dataset in this PoC. A transparent rule is testable and explainable. ML would be justified only after defining labels, costs, drift controls, governance, and a baseline comparison.

What would you change first for production?

Alembic migrations, OAuth/OIDC and authorization, external metrics/traces, Redis/PostgreSQL integration tests, timeouts and pool settings, real workload benchmarks, and a reviewer-feedback data model.

Measured Metrics

These measurements were generated by `scripts/benchmark.py` on the bundled deterministic seed dataset. They are not production claims.

Environment

- Python: 3.13.5
- Platform: Linux-6.18.35-x86_64-with-glibc2.41
- Database profile: `sqlite:///./finance.db`
- Cache profile for this benchmark: in-process TTL cache implementing the same JSON/versioned-key contract as Redis
- Dataset: 8 departments, 24 cost centres, 288 monthly budgets, 30 vendors, 2,330 expenses
- Iterations per latency path: 300

Latency

Path	Mean (ms)	p50 (ms)	p95 (ms)	Min (ms)	Max (ms)
Direct analytical SQL, no cache	1.6647	1.5992	1.7960	1.5513	11.8860

Warm versioned cache	0.0805	0.0778	0.0890	0.0765	0.2329

Measured p50 speed-up: **20.55x**.

Quality and operational metrics

Metric	Measured result	Meaning
Budget reconciliation absolute error	0.00	Aggregated API-query budget equals source budget total
Actual reconciliation absolute error	0.00	Aggregated API-query actual equals source expense total
Warm-cache hit ratio	99.67%	Hits after one priming miss in this benchmark
Exception count	834	Transactions meeting the documented amount and composite-score rule
Exception spend share	56.52%	Share of total spend represented by flagged transactions; this is a triage metric, not confirmed fraud or waste
Welch-test p-value	0.00000000	Synthetic approved vs non-approved amount comparison
Mean-difference 95% CI	[4,836.35, 8,851.39]	Estimated non-approved minus approved mean amount under Welch assumptions

Full-stack benchmark placeholder

Run the Docker profile and capture PostgreSQL plus Redis endpoint latency with:

```
python scripts/http_benchmark.py --base-url http://localhost:8000 --
iterations 300
```

Do not copy the SQLite/in-memory numbers into a production claim. Record machine type, PostgreSQL version, Redis version, concurrent users, dataset size, and cold/warm state.

Sample Json


```
{
  "note": "Generated from the deterministic seed through the actual FastAPI
routes; truncated for readability.",
  "variance_first_3": [
    {
      "department_code": "ENG",
      "department_name": "Engineering",
      "period": "2025-01-01",
      "budget": "583124.18",
      "actual": "646600.3",
      "variance": "63476.12",
      "variance_pct": "10.89"
    },
    {
      "department_code": "ENG",
```

```
      "department_name": "Engineering",
      "period": "2025-02-01",
      "budget": "587770.12",
      "actual": "508268.76",
      "variance": "-79501.36",
      "variance_pct": "-13.53"
    },
    {
      "department_code": "ENG",
      "department_name": "Engineering",
      "period": "2025-03-01",
      "budget": "684439.12",
      "actual": "774054.03",
      "variance": "89614.91",
      "variance_pct": "13.09"
    }
  ],
  "trend_first_3": [
    {
      "period": "2025-01-01",
      "budget": "583124.18",
      "actual": "646600.3",
      "variance": "63476.12",
      "rolling_3m_actual": "646600.3"
    },
    {
      "period": "2025-02-01",
      "budget": "587770.12",
      "actual": "508268.76",
      "variance": "-79501.36",
      "rolling_3m_actual": "1154869.06"
    },
    {
      "period": "2025-03-01",
      "budget": "684439.12",
      "actual": "774054.03",
      "variance": "89614.91",
      "rolling_3m_actual": "1928923.09"
    }
  ],
  "exceptions_first_page": {
    "items": [
      {
        "expense_id": "d0546807-fb3c-4082-821a-202292e57afc",
        "period": "2025-06-01",
        "transaction_date": "2025-06-23",
        "department_code": "FIN",
        "cost_centre_code": "FIN-03",
        "vendor_code": "V004",
        "vendor_name": "Delta Supplies",
        "vendor_risk_tier": "MEDIUM",
        "invoice_number": "INV-202506-000209",
        "amount": "91528.24",
        "approval_status": "REJECTED",
```

```
        "monthly_budget": "145771.38",
        "budget_share_pct": "62.79",
        "exception_score": "122.79",
        "exception_reasons": [
          "approval_rejected",
          "medium_risk_vendor",
          "large_share_of_monthly_budget"
        ]
      },
      {
        "expense_id": "c27a99ee-1e45-4468-a9c6-ff0f94c5b3cf",
        "period": "2025-12-01",
        "transaction_date": "2025-12-05",
        "department_code": "LEGAL",
        "cost_centre_code": "LEGAL-02",
        "vendor_code": "V027",
        "vendor_name": "Acorn Services",
        "vendor_risk_tier": "LOW",
        "invoice_number": "INV-202512-002231",
        "amount": "57909.23",
        "approval_status": "REJECTED",
        "monthly_budget": "86225.7",
        "budget_share_pct": "67.16",
        "exception_score": "117.16",
        "exception_reasons": [
          "approval_rejected",
          "large_share_of_monthly_budget"
        ]
      },
      {
        "expense_id": "56757267-2af9-4925-8df5-741b80f17705",
        "period": "2025-09-01",
        "transaction_date": "2025-09-03",
        "department_code": "FIN",
        "cost_centre_code": "FIN-02",
        "vendor_code": "V007",
        "vendor_name": "Granite Services",
        "vendor_risk_tier": "LOW",
        "invoice_number": "INV-202509-000150",
        "amount": "60354.97",
        "approval_status": "REJECTED",
        "monthly_budget": "111987.5",
        "budget_share_pct": "53.89",
        "exception_score": "103.89",
        "exception_reasons": [
          "approval_rejected",
          "large_share_of_monthly_budget"
        ]
      }
    ],
    "next_cursor":
"eyJzY29yZSI6IjEwMy44OSIsImlkIjoiNTY3NTcyNjctMmFmOS00OTI1LThkZjUtNzQxYjgwZjE
3NzA1In0"
  },
```

```
  "statistics": {
    "approved_n": 1841,
    "non_approved_n": 489,
    "approved_mean": 15821.35,
    "non_approved_mean": 22665.22,
    "mean_difference": 6843.87,
    "ci_95_low": 4836.35,
    "ci_95_high": 8851.39,
    "t_statistic": 6.694,
    "p_value": 0.0,
    "cohen_d": 0.3879,
    "interpretation": "The synthetic data shows a statistically detectable
difference in mean expense amounts between non-approved and approved
transactions. This is association, not causation.",
    "limitations": [
      "The dataset is synthetic and intentionally contains approval-related
patterns.",
      "Expenses are clustered by department, cost centre, period, and
vendor, so observations are not fully independent.",
      "A t-test compares means and does not prove that approval status
causes amount differences.",
      "Heavy tails and outliers can affect both the mean difference and
confidence interval.",
      "Repeated slicing or multiple tests would require multiplicity
control."
    ]
  },
  "operational_metrics_snapshot": {
    "requests_total": 4,
    "errors_total": 0,
    "average_request_latency_ms": 4.72857950001071,
    "cache_hits": 0,
    "cache_misses": 2,
    "cache_hit_ratio": 0.0,
    "status_counts": {
      "200": 4
    }
  }
}
```


Validation Report

Executed checks

Test suite

Command:

```
pytest
```

Result:

```
13 passed in 3.69s
```

Coverage areas include analytics responses, rolling-window logic, stable pagination, idempotent replay, idempotency conflict, validation, cache invalidation, statistics, health, metrics, and optional API-key protection.

Python compilation

Command:

```
python -m compileall -q app scripts tests
```

Result: completed successfully.

Deterministic seed

Generated and loaded:

- 8 departments
- 24 cost centres
- 30 vendors
- 288 monthly budgets
- 2,330 expense transactions

Benchmark

Command:

```
DATABASE_URL=sqlite:///./finance.db REDIS_URL=memory:// \
python scripts/benchmark.py --iterations 300
```

Measured results are stored in `METRICS.md`.

Important qualification

Docker was not available in the execution environment used to create this repository. Therefore:

- PostgreSQL and Redis configuration, DDL, Docker Compose, and benchmark commands are included;
- core behavior was executed through SQLite and the real in-process TTL cache implementation;
- no PostgreSQL/Redis latency number is claimed;
- the full-stack benchmark remains an explicitly marked execution step.